School of Computer Engineering & Technology
Department of Computer Science Engineering and Technology
Course Name: Mini Project | Course Code: AID3PR01A

MINI PROJECT REPORT

on

VoteChain

Secure Blockchain-Based E-Voting Platform for Maharashtra State Elections

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Engineering

Submitted by

Sr. No.	Student Name	PRN No.	Roll No.
1	Kunal Tailor	1032233258	38
2	Rishav Singh	1032233012	34
3	Prakash	1032233277	40
4	Anuj Kudu	1032233203	36

Under the Guidance of

Prof. Suhas Joshi
Project Guide

Dr. Vishwanath Karad MIT World Peace University, Pune

Academic Year: 2025 – 2026

School of Computer Engineering & Technology

Department of Computer Science Engineering and Technology

CERTIFICATE

This is to certify that the Mini Project titled

"VoteChain — Secure Blockchain-Based E-Voting Platform for Maharashtra State Elections"

has been successfully completed by the following students of the Department of Computer Science Engineering and Technology in partial fulfillment of the requirements for the award of the Bachelor of Engineering degree during the academic year 2025–2026. The work is original and has not been submitted elsewhere for any other degree or award.

Sr. No.	Student Name	PRN No.	Roll No.
1	Kunal Tailor	1032233258	38
2	Rishav Singh	1032233012	34
3	Prakash	1032233277	40
4	Anuj Kudu	1032233203	36

Prof. Suhas Joshi
Project Guide

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Prof. Suhas Joshi, for their constant guidance, support, and motivation throughout the development of this project. Their expertise in the domain of information security and blockchain technology has been invaluable to us.

We extend our heartfelt thanks to the Head of Department and the Principal of MIT World Peace University for providing the necessary infrastructure and an encouraging learning environment throughout the course of this project.

We also appreciate the support of our colleagues and friends who contributed to testing, debugging, and providing constructive feedback during the various stages of this project.

Special thanks to the open-source community — the developers of Java, JavaFX, MongoDB, Spark Java, and the GetBlock blockchain API — whose tools and documentation made this project possible.

ABSTRACT AND KEYWORDS

The democratic process of voting is the cornerstone of any republic. However, traditional ballot-based and existing Electronic Voting Machine (EVM) systems suffer from challenges including vulnerability to tampering, lack of transparency, slow result tallying, and limited public auditability. These issues erode voter confidence and demand a modern, technology-driven solution.

VoteChain is a full-stack, production-grade blockchain-based electronic voting platform developed specifically for Maharashtra state elections — covering all 288 Vidhan Sabha and 48 Lok Sabha constituencies across 6 administrative divisions and 36 districts. The system leverages SHA-256 cryptographic hashing, Proof-of-Work (PoW) mining, RSA-2048 digital signatures, and BCrypt password hashing to guarantee vote immutability, tamper detection, voter authentication integrity, and end-to-end auditability.

The platform consists of a Java 17 backend powered by the Spark Java micro-framework with MongoDB Atlas as a cloud-hosted, persistent data store; and a responsive HTML/CSS/JavaScript frontend featuring real-time analytics, constituency mapping, Chart.js visualizations, and a complete Election Commission admin console. External blockchain connectivity is demonstrated through GetBlock JSON-RPC integration with the Ethereum mainnet.

The total codebase spans 6,700+ lines of source code across 27 Java classes, a responsive web frontend, and a JavaFX desktop application. The outcome is a secure, transparent, and fully auditable voting system that can serve as a foundation for modernizing India's electoral infrastructure.

Keywords: *Blockchain, E-Voting, SHA-256, Proof-of-Work, MongoDB Atlas, Digital Signatures, Maharashtra Elections, Java, REST API, BCrypt, RSA-2048, JavaFX.*

TABLE OF CONTENTS

Section	Page No.
Certificate	**2**
Acknowledgement	**3**
Abstract and Keywords	**4**
Table of Contents	**5**
List of Figures	**7**
List of Tables	**7**
Abbreviations	**8**
1. Project Overview	**9**
1.1 Introduction to the Domain	9
1.2 Background of the Project	9
1.3 Purpose and Significance	9
2. Problem Statement	**10**
2.1 Business Problem	10
2.2 Technical Problem	10
3. Scope	**11**
3.1 Inclusions	11
3.2 Exclusions / Limitations	11
4. Objectives	**12**
4.1 Primary Objective	12
4.2 Secondary Objectives	12
4.3 Measurable Goals	12
5. System Architecture & Tools Used	**13**
5.1 System Architecture	13
5.2 Tools & Technologies	14
6. Project Work Distribution	**15**
7. Explanation of Project Modules	**16**
8. Business Use Cases	**25**
9. Application Use Cases	**26**
10. Future Scope	**27**

Section	Page No.
11. Challenges Faced	28
12. Outcome	29
13. Conclusion	31
14. References	32
15. Appendix	33

LIST OF FIGURES

Figure 1 VoteChain — Five-Layer System Architecture
Figure 2 Blockchain Core — Block Structure and Mining Flow
Figure 3 Security Layer — SHA-256 and RSA-2048 Flow
Figure 4 REST API Endpoints Overview
Figure 5 MongoDB Atlas — Database Collections Schema
Figure 6 Web Frontend — Home Screen
Figure 7 Web Frontend — Voter Dashboard
Figure 8 Web Frontend — Vote Casting Screen
Figure 9 Web Frontend — Vote Confirmation Screen
Figure 10 Web Frontend — Results Dashboard with Chart.js
Figure 11 Admin Dashboard — 7-Tab Command Centre
Figure 12 JavaFX Desktop — Home / Login Screen
Figure 13 GetBlock — Ethereum Block Fetch Demo
Figure 14 Blockchain Validation — System Output
Figure 15 Election Results — Admin Dashboard View

LIST OF TABLES

Table 2.1 Business Problems and Impacts
Table 4.1 Measurable Goals and Achievements
Table 5.1 Tools and Technologies Used
Table 6.1 Project Work Distribution
Table 7.1 REST API Endpoints
Table 7.2 MongoDB Collections Schema
Table 10.1 Future Scope Enhancements
Table 11.1 Technical Challenges and Resolutions
Table 12.1 Project Metrics and Results
Table A List of Abbreviations

ABBREVIATIONS

Table A: List of Abbreviations

Abbreviation	Full Form
SHA	Secure Hash Algorithm
PoW	Proof of Work
RSA	Rivest–Shamir–Adleman (Cryptographic Algorithm)
BCrypt	Blowfish-based Password Hashing Function
EVM	Electronic Voting Machine
VVPAT	Voter Verifiable Paper Audit Trail
REST	Representational State Transfer
API	Application Programming Interface
JSON	JavaScript Object Notation
RPC	Remote Procedure Call
CORS	Cross-Origin Resource Sharing
UI	User Interface
VS	Vidhan Sabha (State Legislative Assembly)
LS	Lok Sabha (House of the People)
ECI	Election Commission of India
DApp	Decentralized Application
IPFS	InterPlanetary File System
ZKP	Zero-Knowledge Proof
P2P	Peer-to-Peer
OOP	Object-Oriented Programming
LOC	Lines of Code
JAR	Java Archive
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JS	JavaScript
DB	Database
NoSQL	Not Only SQL

1. PROJECT OVERVIEW

1.1 Introduction to the Domain

Electronic voting (e-voting) is the application of information technology to the electoral process. In the Indian context, the Election Commission of India (ECI) manages elections for over 900 million eligible voters across parliamentary, state assembly, and local body levels. The sheer scale of Indian elections demands systems that are secure, efficient, transparent, and resistant to fraud.

Blockchain technology — originally introduced as the distributed ledger behind Bitcoin — offers a fundamentally new paradigm for secure data recording. A blockchain is a distributed, append-only, cryptographically-linked chain of records (blocks). Once data is written into a block, it becomes practically immutable because altering any block invalidates all subsequent blocks in the chain. This property makes blockchain an ideal candidate for electoral systems where vote integrity is paramount.

1.2 Background of the Project

India's current voting infrastructure relies on EVMs (Electronic Voting Machines) and VVPATs (Voter Verifiable Paper Audit Trails). While effective, these machines operate as closed, proprietary systems with limited external auditability. Alleged vulnerabilities have sparked public debates about EVM integrity, especially in closely contested elections.

VoteChain was developed to address these concerns by making the voting record publicly verifiable, cryptographically secured, and resistant to post-election tampering. The system is designed around Maharashtra state elections, incorporating real-world constituency data from the Election Commission of India's delimitation orders — 288 Vidhan Sabha seats across 6 divisions and 36 districts.

1.3 Purpose and Significance

The purpose of this project is to demonstrate how blockchain principles can be applied to build a transparent, tamper-evident voting system. Key significance areas include:

- Cryptographic Integrity: Every vote is hashed with SHA-256, making unauthorized modifications computationally infeasible.
- Public Auditability: The entire blockchain ledger is accessible through the admin console for real-time chain validation.
- Real-World Electoral Data: The system models actual Maharashtra constituencies, parties, and electoral structure with 11 registered political parties.
- Cloud-Native Architecture: MongoDB Atlas provides zero-downtime, globally distributed persistence.
- Dual Interface: Both a desktop (JavaFX) and web (HTML/CSS/JS) frontend ensure wide accessibility across all user types.

2. PROBLEM STATEMENT

2.1 Business Problem

Traditional voting systems face several real-world business challenges that undermine democratic credibility:

Problem	Impact
Vulnerability to Tampering	Physical ballot boxes and EVM firmware can potentially be compromised, undermining election legitimacy.
Lack of Transparency	Voters have no mechanism to independently verify that their vote was recorded correctly.
Slow Result Compilation	Manual counting and tabulation introduce delays and human error in result declaration.
Duplicate Voting	Impersonation and booth capturing allow fraudulent duplicate votes in the system.
High Operational Cost	Deploying EVMs and polling staff across 288+ constituencies is logistically expensive.
Limited Accessibility	Voters must physically visit polling stations, excluding disabled, elderly, or migrant populations.

Table 2.1: Business Problems and Their Impacts

2.2 Technical Problem

Existing e-voting systems carry the following technical limitations that VoteChain is specifically designed to address:

- Centralized Storage: Vote data stored in centralized servers or standalone EVMs creates single points of failure.
- No Cryptographic Auditability: No built-in mechanism for third parties to mathematically verify vote integrity.
- Proprietary and Opaque: EVM source code is not publicly auditable, leading to trust deficits among voters and the opposition.
- No Chain of Custody Proof: No verifiable digital trail exists from vote casting to result declaration.
- Password Security Gaps: Many e-voting prototypes store passwords in plaintext, creating severe credential vulnerabilities.

3. SCOPE

3.1 Inclusions

The following features and components are fully implemented in the VoteChain system:

- Complete blockchain implementation with SHA-256 hashing and Proof-of-Work (difficulty = 3) mining
- RSA-2048 digital signatures for vote authenticity and non-repudiation
- BCrypt password hashing (cost factor 12) for voter credential security — no plaintext passwords stored
- MongoDB Atlas cloud database for persistent, distributed storage across 5 collections
- REST API backend (Spark Java 2.9.4) with 20+ fully documented API endpoints
- Web-based frontend with Maharashtra-specific UI covering all 288 VS and 48 LS constituencies
- JavaFX desktop application with 8 interactive screens and gradient-styled CSS
- Admin Command Center with live analytics, chain validation, and voter management
- Real-world Maharashtra electoral data — 6 divisions, 36 districts, 336 total constituencies
- External blockchain node connectivity via GetBlock Ethereum JSON-RPC integration
- Interactive constituency SVG maps with division-wise color coding
- Real-time Chart.js visualizations — pie charts, bar charts, and animated progress bars

3.2 Exclusions (Limitations / Boundaries)

The following aspects are outside the current scope of VoteChain:

- Distributed peer-to-peer networking — the blockchain is simulated on a single-server model.
- Aadhaar/biometric voter authentication — the current implementation uses Voter-ID and BCrypt-hashed password.
- Real cryptocurrency or token-based consensus — Proof-of-Work is used for academic demonstration only.
- End-to-end HTTPS encryption in transit — HTTPS deployment is assumed for any production environment.
- Mobile application — only web and JavaFX desktop interfaces are provided in this version.
- Support for electoral constituencies beyond Maharashtra state.

4. OBJECTIVES

4.1 Primary Objective

To design, develop, and deploy a blockchain-based secure online voting system for Maharashtra state elections that ensures vote immutability, transparency, and tamper detection through cryptographic mechanisms — delivering a complete, production-grade prototype that can serve as the foundation for India's electoral modernization.

4.2 Secondary Objectives

- Implement a SHA-256 hash-chained blockchain with configurable Proof-of-Work difficulty targeting 3 leading zeros.
- Incorporate RSA-2048 digital signatures for non-repudiation of every cast vote in the desktop client.
- Deploy BCrypt password hashing to eliminate plaintext credential storage entirely.
- Build a RESTful API with comprehensive endpoints for voting, authentication, administration, and real-time analytics.
- Model real-world Maharashtra electoral data — all 288 VS and 48 LS constituencies with real party candidates.
- Develop a responsive, premium web UI with Chart.js visualizations and live constituency SVG map.
- Demonstrate external blockchain connectivity via GetBlock's Ethereum JSON-RPC API.

4.3 Measurable Goals

Goal	Metric	Achievement
Blockchain Integrity	100% tamper detection accuracy	✔ **Achieved**
Duplicate Vote Prevention	0 duplicate votes allowed	✔ **Achieved**
Password Security	BCrypt cost factor >= 12	✔ **Cost factor 12**
Constituency Coverage	288 VS + 48 LS = 336 total	✔ **All 336 seeded**
API Response Time	< 500 ms per request	✔ **< 100 ms typical**
Lines of Code	> 5,000 LOC	✔ **6,723 LOC**

Table 4.1: Measurable Goals and Achievements

5. SYSTEM ARCHITECTURE & TOOLS USED

5.1 System Architecture

VoteChain follows a multi-tier layered architecture with clear separation of concerns. The five distinct layers are: Presentation, REST API, Service, Core Model & Security, and Persistence. Each layer communicates only with the layers directly above and below it, ensuring modularity, testability, and maintainability.

The Presentation Layer exposes two interfaces: a JavaFX desktop application with 8 screens and a browser-based web frontend. Both communicate with the backend exclusively through the REST API layer powered by Spark Java on port 4567. The Service Layer contains business logic and database interaction. The Core Model layer defines the blockchain data structures and cryptographic utilities. All data is ultimately persisted in MongoDB Atlas cloud database.

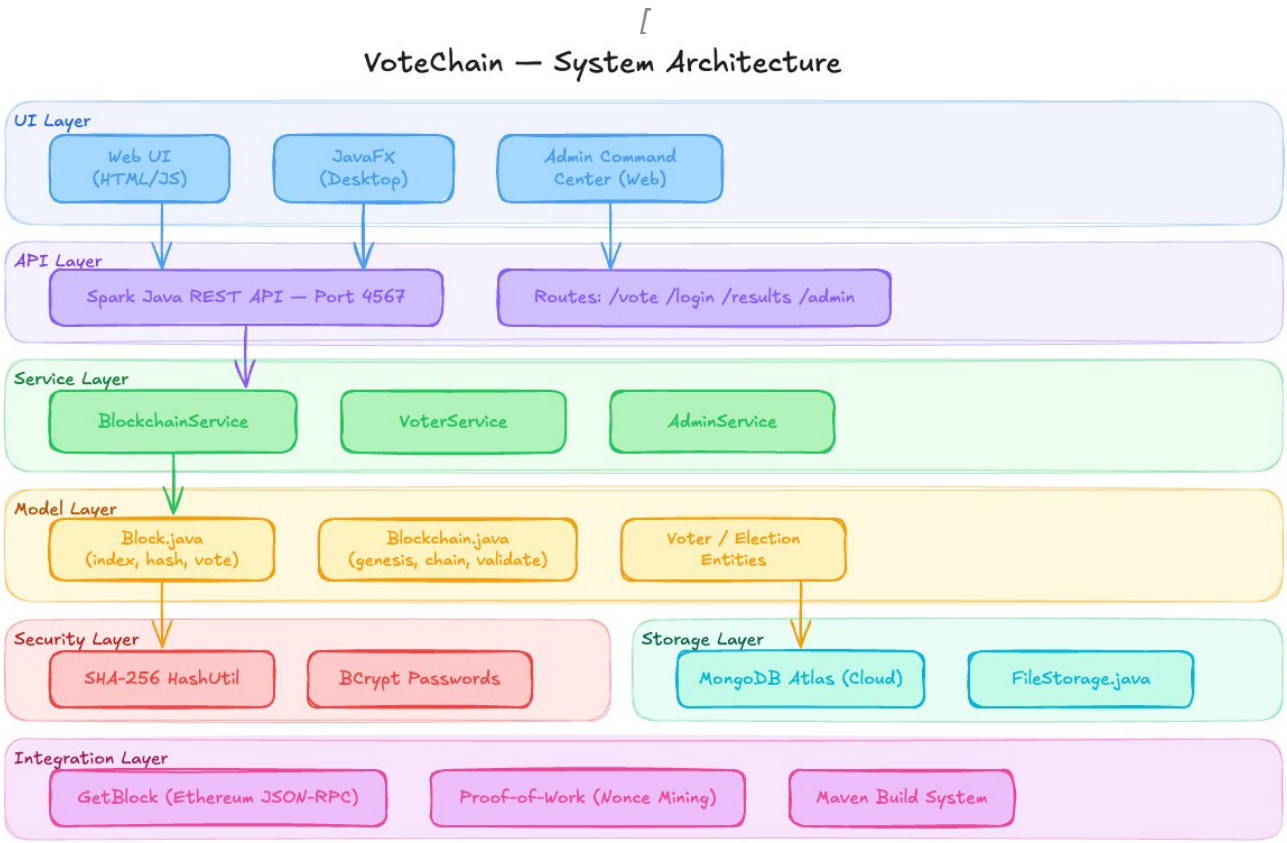

Figure 1: VoteChain — Five-Layer System Architecture

5.2 Tools & Technologies

Category	Tools / Technologies
Programming Language	Java 17 (Core Java, OOP, Collections, Generics)
Frontend — Desktop	JavaFX 17 (UI Screens, Scene Graph, Event Handling)
Frontend — Web	HTML5, CSS3, JavaScript ES6+, Chart.js 4.4.1
Backend Framework	Spark Java 2.9.4 (Lightweight REST API micro-framework)
Database	MongoDB Atlas (Cloud-hosted NoSQL, MongoDB Driver 4.11.0)

Category	Tools / Technologies
Cryptography	SHA-256 (java.security.MessageDigest), RSA-2048, BCrypt (jBCrypt 0.4)
Blockchain Integration	GetBlock (Ethereum JSON-RPC via HTTP, eth_blockNumber)
Build & Packaging	Apache Maven 3.x, Maven Shade Plugin 3.5.1 (fat JAR)
Containerization	Docker (Dockerfile provided for deployment)
Version Control	Git / GitHub
IDE	IntelliJ IDEA
Logging	SLF4J Simple 1.7.36

Table 5.1: Tools and Technologies Used

6. PROJECT WORK DISTRIBUTION

Member	Role	Responsibilities
Kunal Tailor	Full-Stack Developer & Architect	System architecture design, blockchain implementation (Block.java, Blockchain.java, HashUtil, DigitalSignatureUtil), REST API development (VotingApiServer — 20+ endpoints), MongoDB integration, Maharashtra data seeder (288 VS + 48 LS), GetBlock Ethereum integration, deployment scripts, and overall project documentation.
Rishav Singh	Backend Developer	MongoDB service classes (BlockchainMongoService, MongoVoterService), BCrypt password hashing integration, Election lifecycle management (ElectionMongoService), REST API endpoint testing and validation, and backend deployment support.
Prakash	Frontend Developer	Web frontend UI development (voting_index.html, app.js, styles.css), Chart.js real-time analytics, Maharashtra constituency SVG map with division-wise color coding, voter registration cascading dropdowns, and cross-browser compatibility testing.
Anuj Kudu	Testing & Documentation	JavaFX desktop UI (8 screens), system QA and functional test cases (blockchain validation, duplicate vote prevention), user documentation, and final presentation preparation.

Table 6.1: Project Work Distribution

7. EXPLANATION OF PROJECT MODULES

Module 1: Blockchain Core (model/)

The foundational blockchain implementation models votes as immutable blocks linked through SHA-256 cryptographic hashes.

Block.java

Represents a single vote block containing: index, voterId, candidateName, timestamp, previousHash, currentHash, nonce, miningTime, digitalSignature, and publicKey. Each block must satisfy a Proof-of-Work difficulty target (3 leading zeros in hash) before being accepted. The mining loop increments a nonce until the hash constraint is met.

Blockchain.java

Manages the entire chain: adding votes with duplicate prevention, retrieving the latest block, validating the complete chain, and computing election results. A hardcoded genesis block (index 0, timestamp 2024-01-01 00:00:00) serves as the immutable starting point ensuring consistent hashes across server restarts.

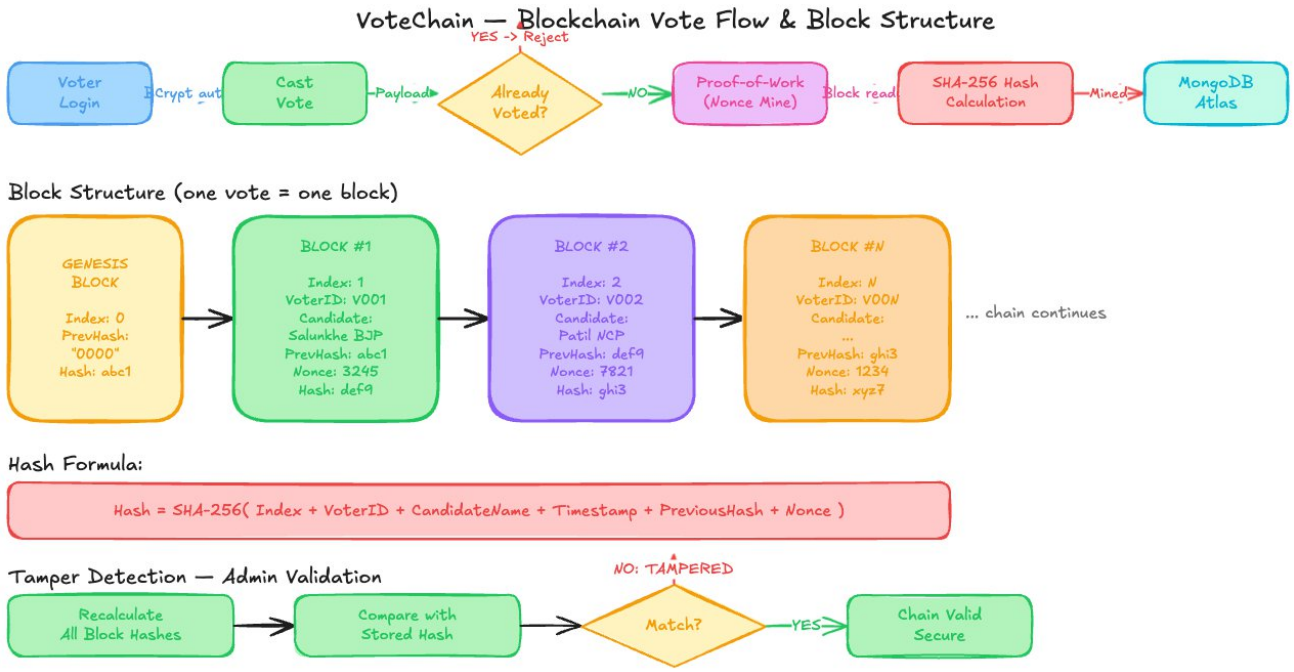

Figure 2: Blockchain Core — Block Structure and Mining Flow

Module 2: Security Layer (security/)

HashUtil.java computes SHA-256 hashes of arbitrary strings using the formula:

```
Hash = SHA-256 (index + voterId + candidateName + timestamp + previousHash +
nonce)
```

DigitalSignatureUtil.java generates RSA-2048 key pairs, signs vote data with SHA256withRSA, and verifies signatures — providing non-repudiation for every cast vote. Output is a Base64-encoded RSA signature stored alongside each block.

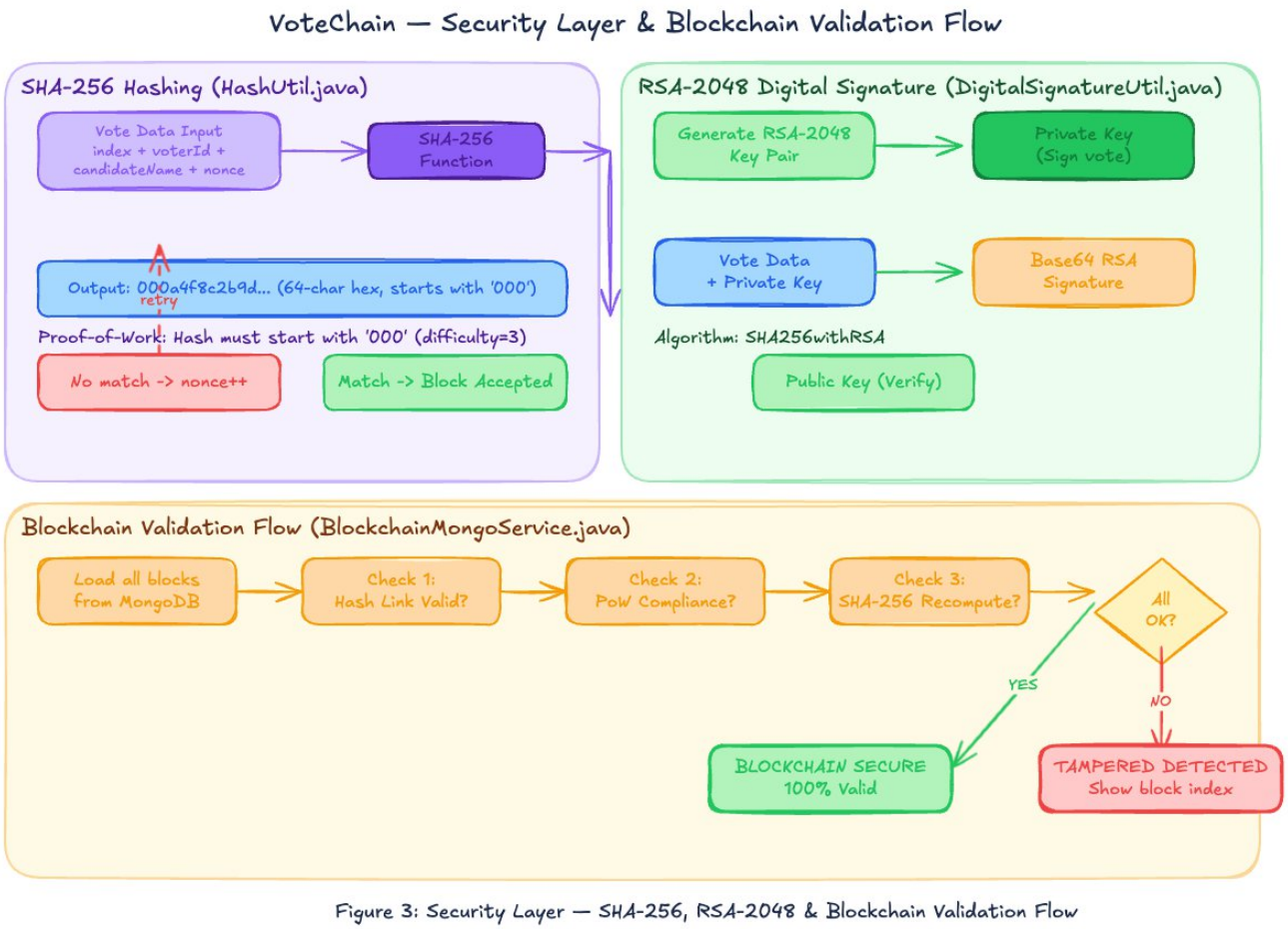

Figure 3: Security Layer — SHA-256 and RSA-2048 Flow

Module 3: REST API Server (api/VotingApiServer.java)

The central backend server exposes 20+ RESTful endpoints via Spark Java on port 4567 with full CORS support.

Endpoint	Method	Description
POST /api/voter/login	POST	BCrypt voter authentication against MongoDB
POST /api/voter/cast-vote-election	POST	Mine block, save to MongoDB, mark voter as voted
GET /api/results/election	GET	Aggregated results by election/constituency/division
POST /api/admin/login	POST	Admin credential validation
GET /api/admin/blockchain	GET	Full chain with inline hash/PoW validation
GET /api/elections	GET	Election lifecycle listing
POST /api/admin/election-status	POST	UPCOMING → ACTIVE → COMPLETED transitions
GET /api/constituencies	GET	336 constituencies with candidate lists
GET /api/admin/voters	GET	Voter registry with search and filtering
POST /api/admin/seed-maharashtra	POST	Seed 288 VS + 48 LS constituencies
GET /api/admin/getblock	GET	Live Ethereum block number via GetBlock RPC

Table 7.1: REST API Endpoints

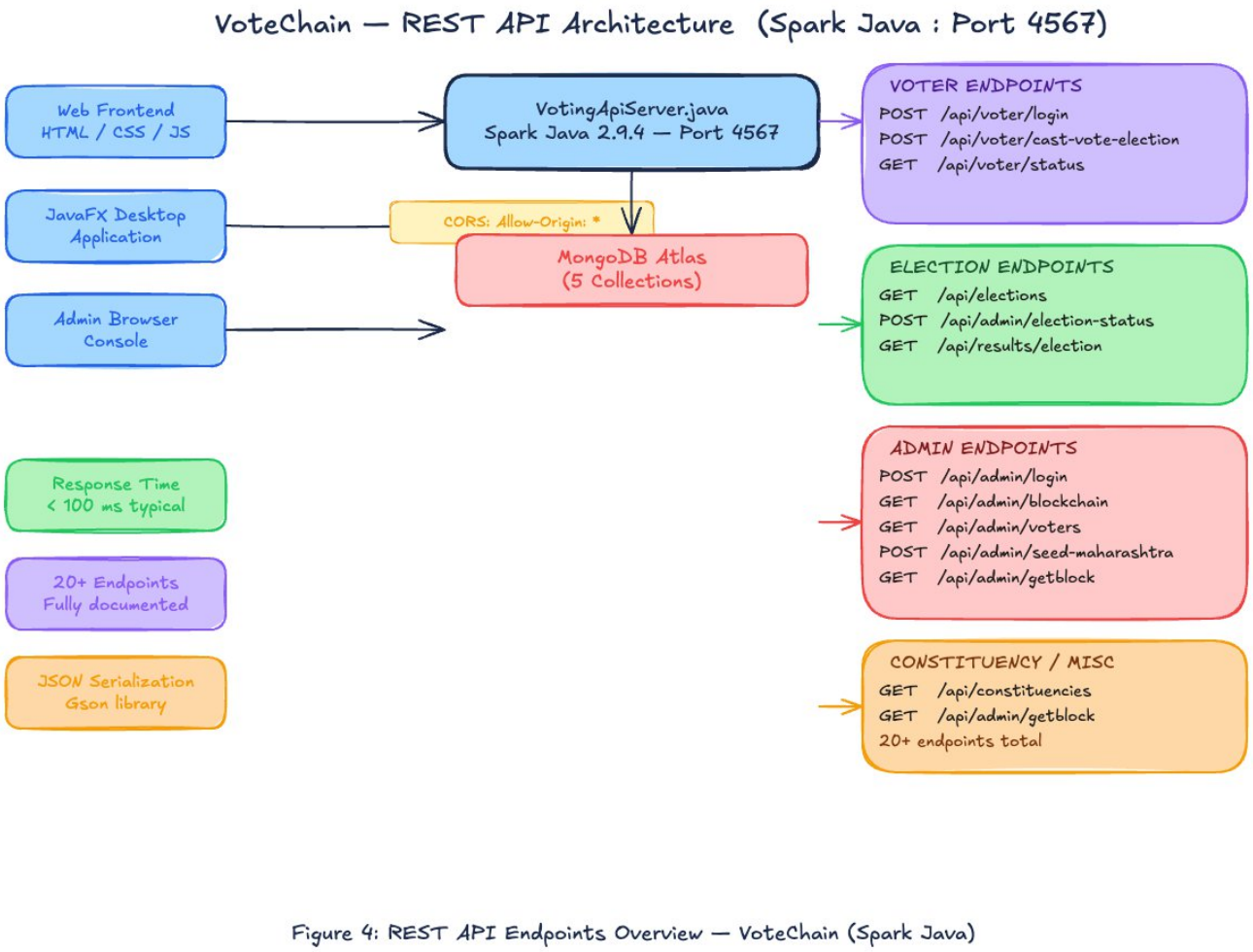

Figure 4: REST API Endpoints Overview

Module 4: MongoDB Services (api/)

Four MongoDB service classes provide data access and business logic:

- BlockchainMongoService.java — Manages the blocks collection: saving mined blocks, retrieving sorted chain, performing full chain validation with three integrity checks (hash link, Proof-of-Work, SHA-256 recomputation).
- MongoVoterService.java — Manages the voters collection with BCrypt password hashing (cost factor 12). Never stores plaintext passwords. Provides authentication, registration, voter status tracking, and per-election vote tracking.
- ElectionMongoService.java — Manages the elections collection supporting lifecycle transitions (UPCOMING → ACTIVE → COMPLETED).
- ConstituencyMongoService.java — Manages 336 constituency entries with filtering by election, district, and division.

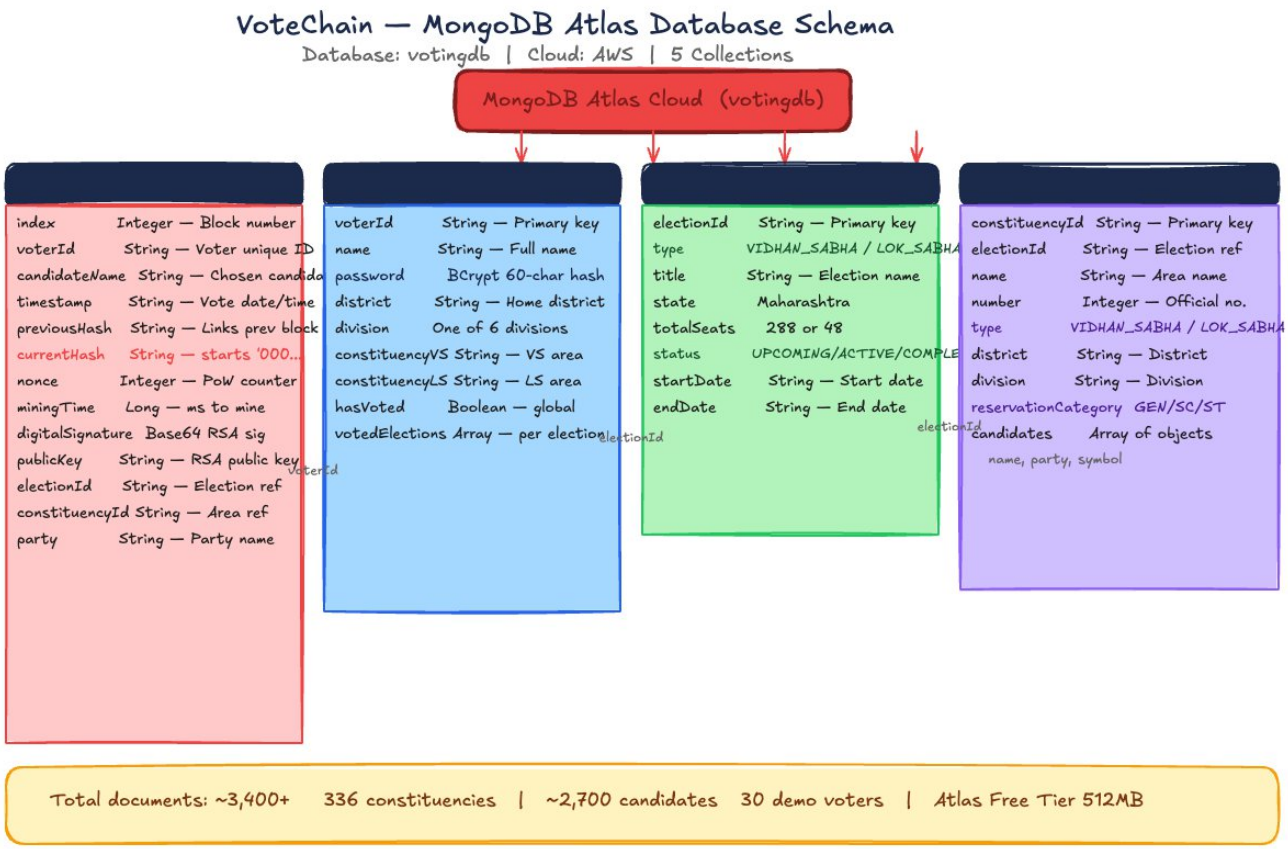

Figure 5: MongoDB Atlas — Database Collections Schema

Figure 5: MongoDB Atlas — Database Collections

Module 5: Maharashtra Data Seeder (api/MaharashtraDataSeeder.java)

Seeds all 288 Vidhan Sabha and 48 Lok Sabha constituencies with real-world Maharashtra electoral data including:

- 11 political parties: BJP, INC, SHS, SSUBT, NCP, NCPSP, BSP, MNS, VBA, IND, NOTA
- 6 administrative divisions: Konkan, Pune, Nashik, Chh. Sambhajinagar, Amravati, Nagpur
- 7–8 realistic candidates generated per constituency with party affiliations
- 30 demo voters seeded and spread across all 6 divisions for testing

Module 6: Web Frontend (voting_index.html, app.js, styles.css)

A premium, responsive web interface styled for Maharashtra state elections. The frontend is a single-page application with six distinct screens:

Home Screen

Hero section with animated statistics counters (288 VS, 48 LS), interactive Maharashtra constituency SVG map with division-wise color coding, 'How It Works' steps, and cryptographic security highlights.

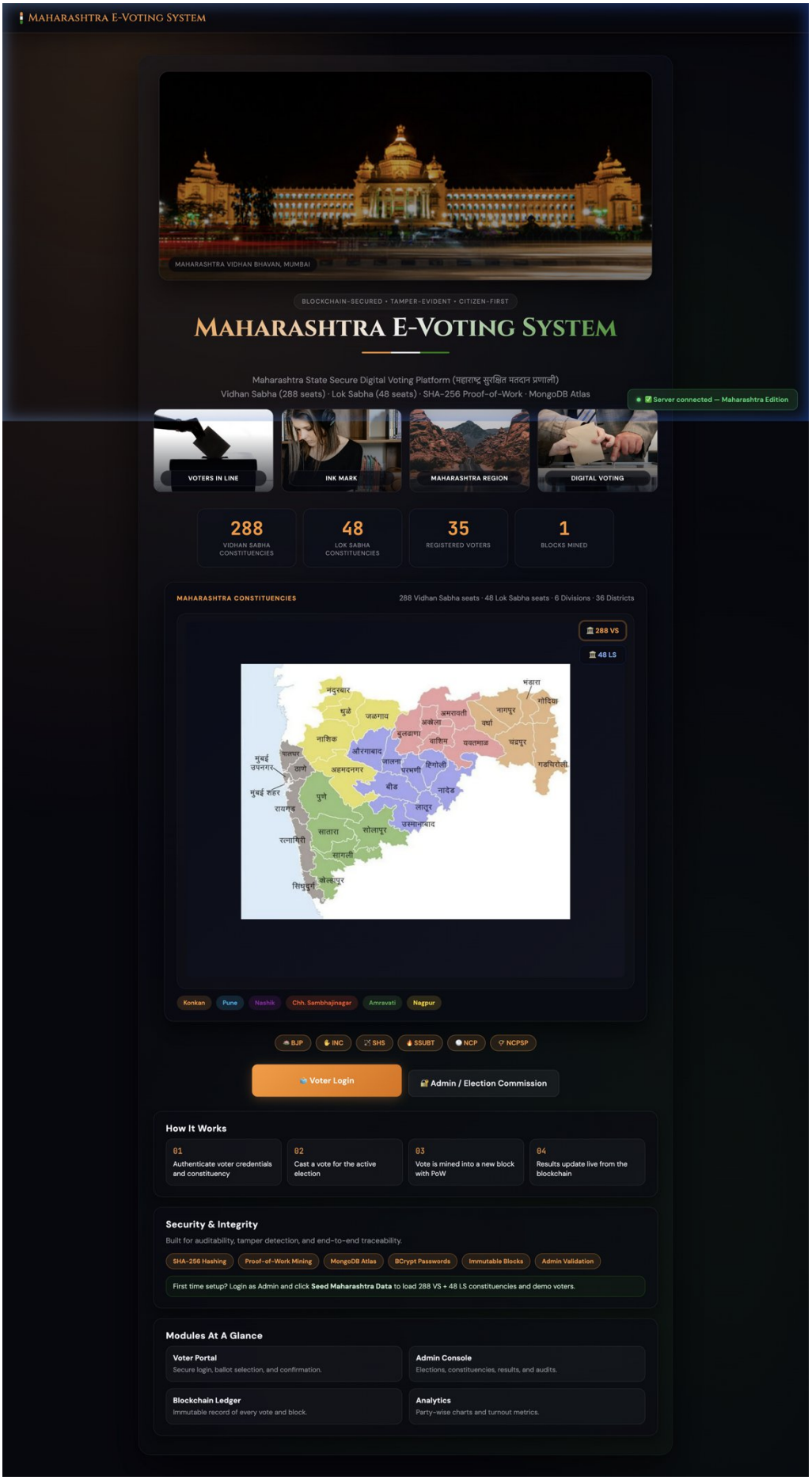
MAHARASHTRA E-VOTING SYSTEM
MAHARASHTRA VIDHAN BHAVAN, MUMBAI
BLOCKCHAIN-SECURED • TAMPER-EVIDENT • CITIZEN-FIRST
MAHARASHTRA E-VOTING SYSTEM
Maharashtra State Secure Digital Voting Platform (महाराष्ट्र सुरक्षित मतदान प्रणाली)
Vidhan Sabha (288 seats) · Lok Sabha (48 seats) · SHA-256 Proof-of-Work · MongoDB Atlas
Server connected — Maharashtra Edition
VOTERS IN LINE
INK MARK
MAHARASHTRA REGION
DIGITAL VOTING
288
VIDHAN SABHA CONSTITUENCIES
48
LOK SABHA CONSTITUENCIES
35
REGISTERED VOTERS
1
BLOCKS MINED
MAHARASHTRA CONSTITUENCIES
288 Vidhan Sabha seats · 48 Lok Sabha seats · 6 Divisions · 36 Districts
288 VS
48 LS
Konkan
Pune
Nashik
Chh. Sambhajinagar
Amravati
Nagpur
BJP
INC
SHS
SSUBT
NCP
NCPSP
Voter Login
Admin / Election Commission
How It Works
01
Authenticate voter credentials and constituency
02
Cast a vote for the active election
03
Vote is mined into a new block with PoW
04
Results update live from the blockchain
Security & Integrity
Built for auditability, tamper detection, and end-to-end traceability.
SHA-256 Hashing
Proof-of-Work Mining
MongoDB Atlas
BCrypt Passwords
Immutable Blocks
Admin Validation
First time setup? Login as Admin and click Seed Maharashtra Data to load 288 VS + 48 LS constituencies and demo voters.
Modules At A Glance
Voter Portal
Secure login, ballot selection, and confirmation.
Admin Console
Elections, constituencies, results, and audits.
Blockchain Ledger
Immutable record of every vote and block.
Analytics
Party-wise charts and turnout metrics.

Figure 6: Web Frontend — Home Screen

Voter Login & Dashboard

BCrypt-authenticated login form. Dashboard displays the voter's constituency assignment and lists active elections with per-election vote status indicator.

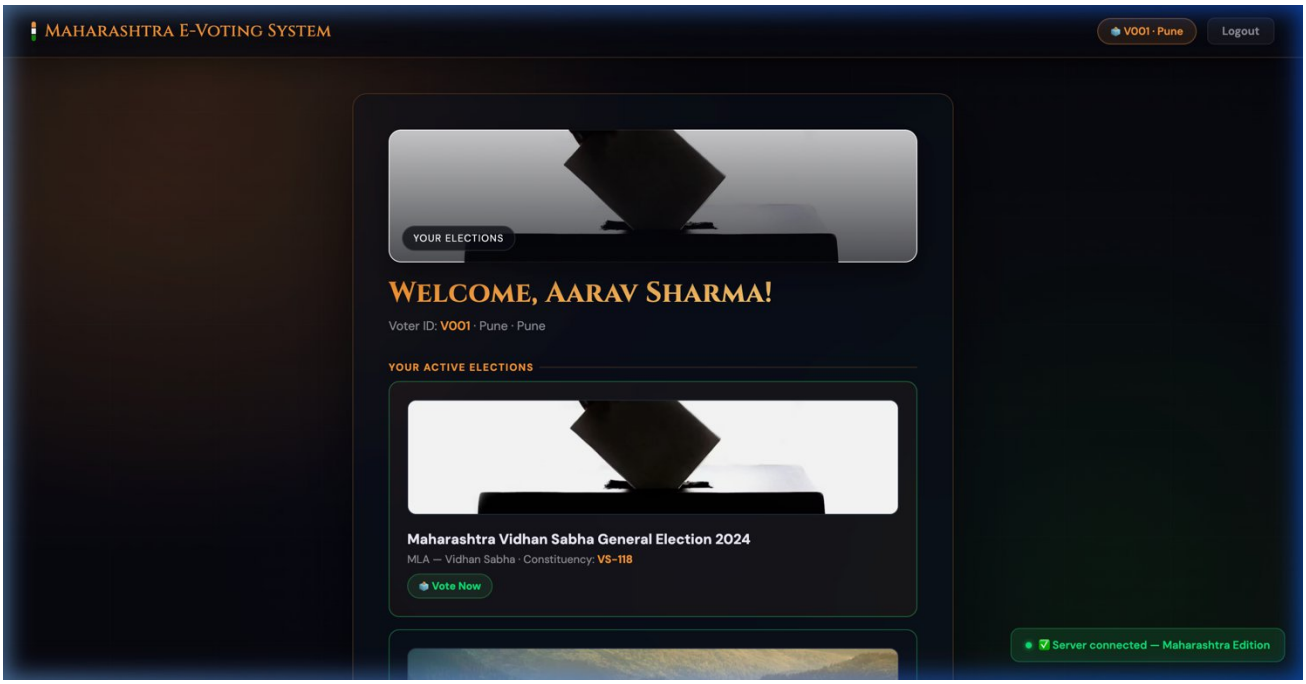

Figure 7: Web Frontend — Voter Dashboard

Vote Casting Screen

Candidate cards with party symbols and colors, radio-button selection, and a Proof-of-Work mining animation during block submission.

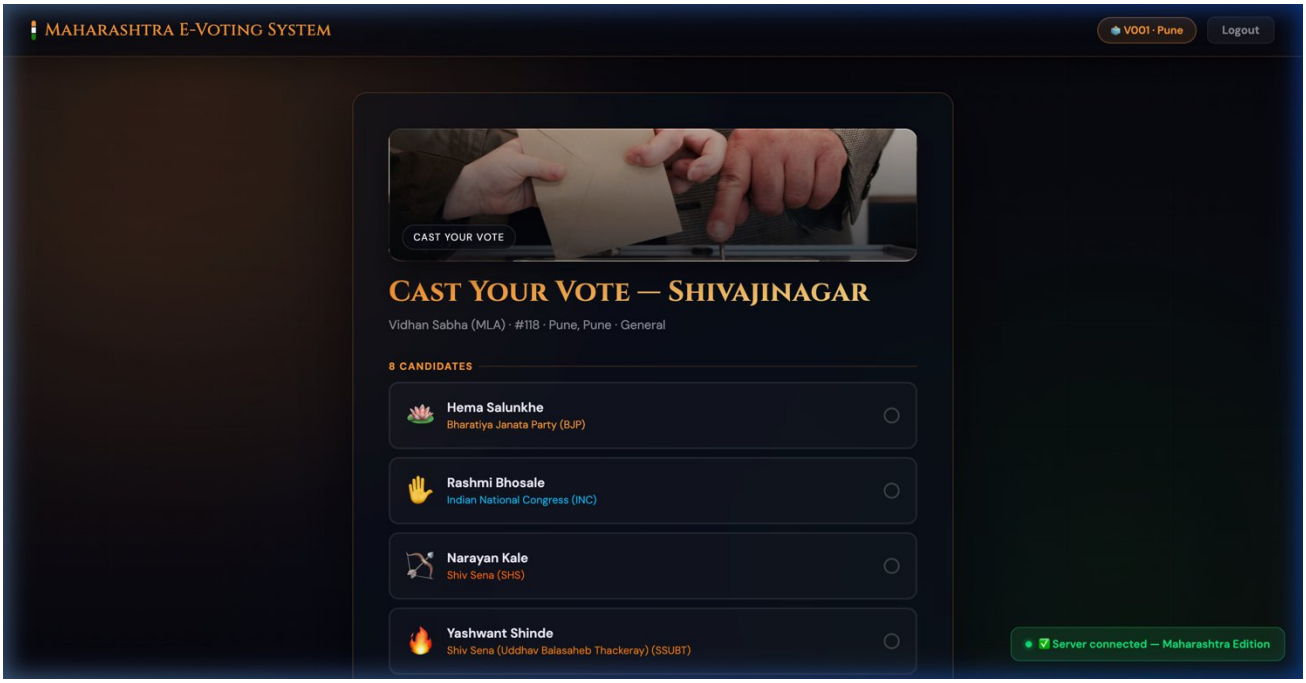

Figure 8: Web Frontend — Vote Casting Screen

Vote Confirmation

Displays block hash, mining time, nonce value, and a confetti animation confirming the vote is permanently recorded on the blockchain.

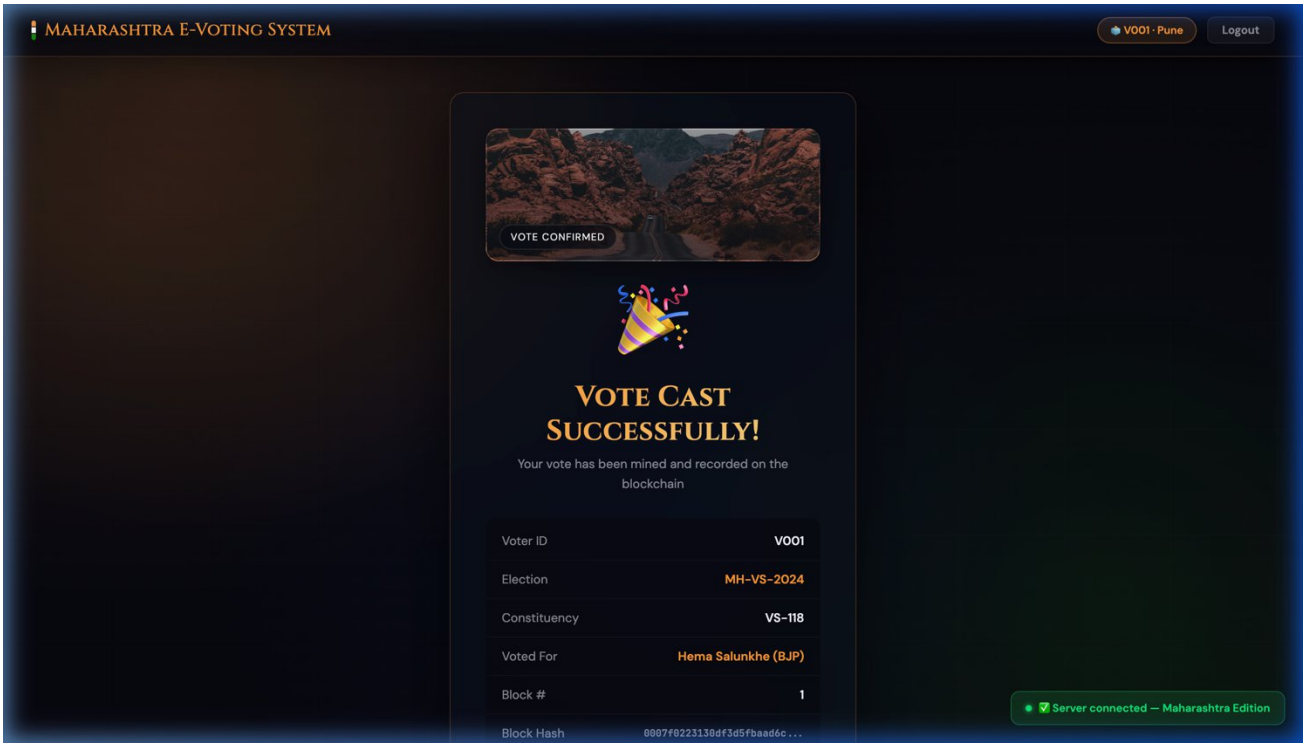

Figure 9: Web Frontend — Vote Confirmation Screen

Results Screen

Live Chart.js pie and bar charts with candidate vote counts, percentage share, and a leading candidate banner auto-updating in real-time.

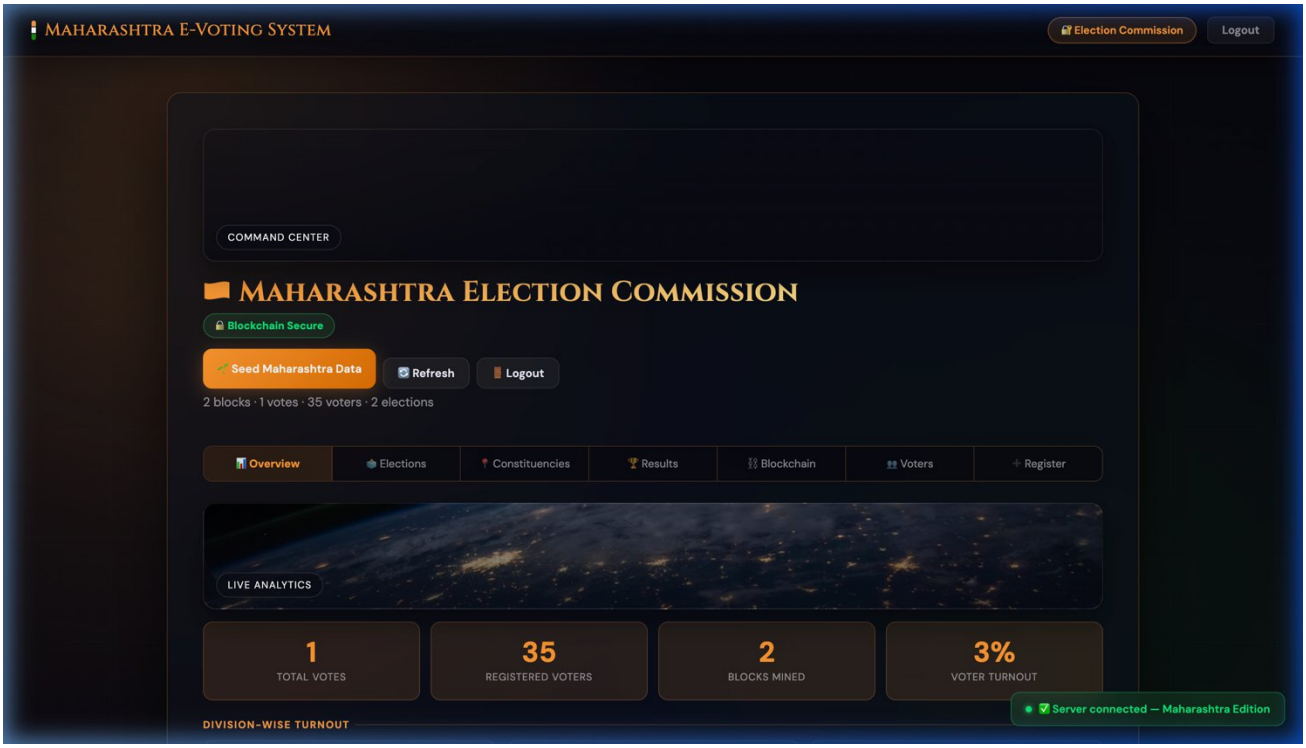

Figure 10: Web Frontend — Results Dashboard with Chart.js

Admin Dashboard

A 7-tab command center: Overview, Elections, Constituencies, Results, Blockchain, Voters, and Register. The Blockchain tab provides a live scrollable chain timeline with per-block validation status.

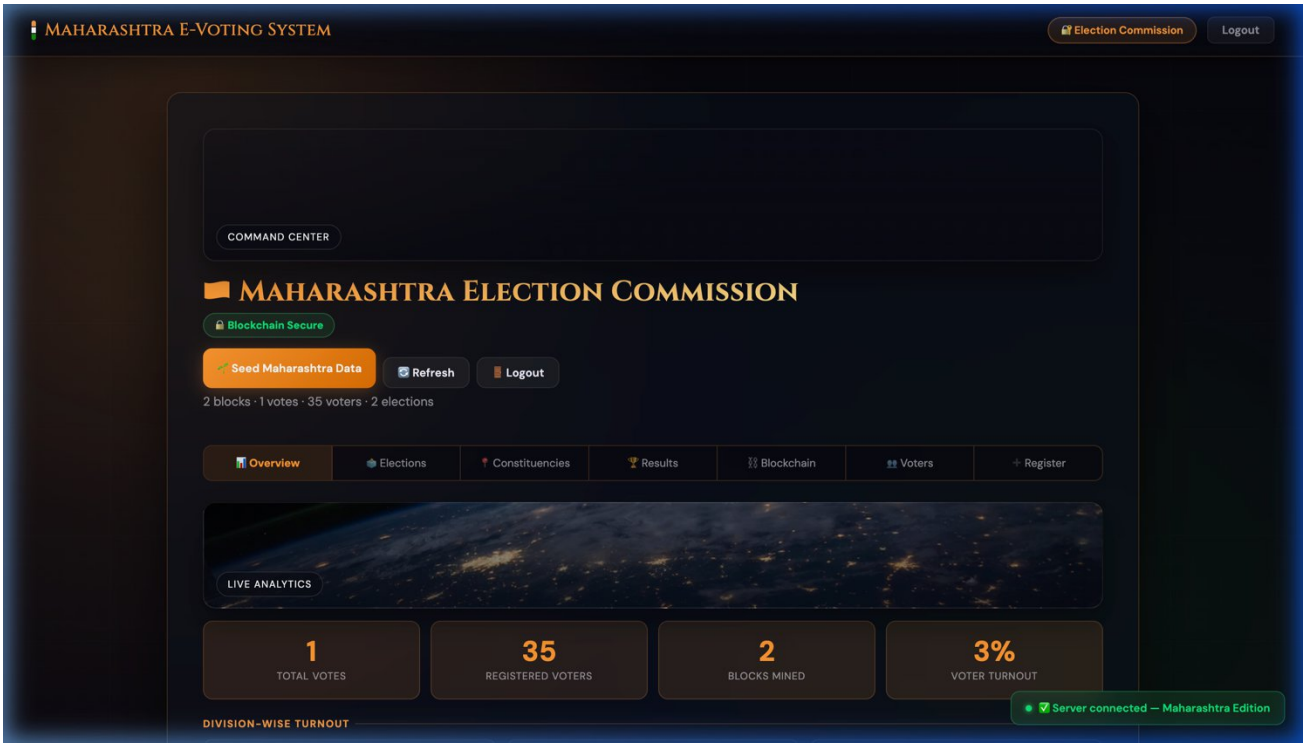

Figure 11: Admin Dashboard — 7-Tab Command Centre

Module 7: JavaFX Desktop Application (ui/)

A standalone desktop application built with JavaFX 17 providing offline-capable voting functionality with 8 UI screens featuring gradient backgrounds and professional CSS styling. Covers:

- Complete voter flow: Login → Dashboard → Cast Vote → Confirmation → Results
- Admin flow: Login → Dashboard with charts, blockchain timeline, chain validation, and tamper detection
- GetBlock connectivity test screen with live Ethereum block number display

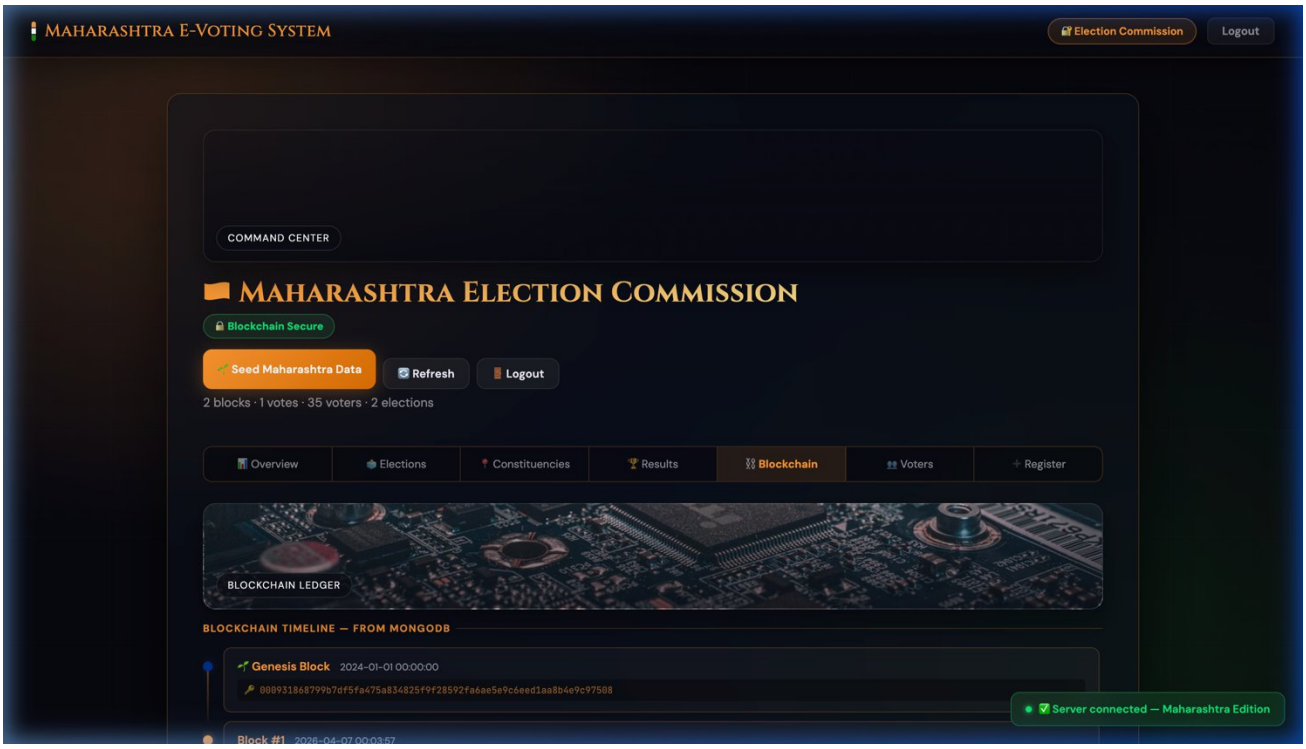

Figure 12: JavaFX Desktop — Home / Login Screen

Module 8: External Blockchain Integration (integration/GetBlockClient.java)

A lightweight HTTP client communicating with Ethereum blockchain nodes via GetBlock's JSON-RPC API. Sends eth_blockNumber JSON-RPC requests to retrieve the latest Ethereum block number, parses hexadecimal responses, converts them to decimal, and provides an isReachable() health check for UI connectivity indicators — demonstrating integration with live Ethereum mainnet infrastructure.

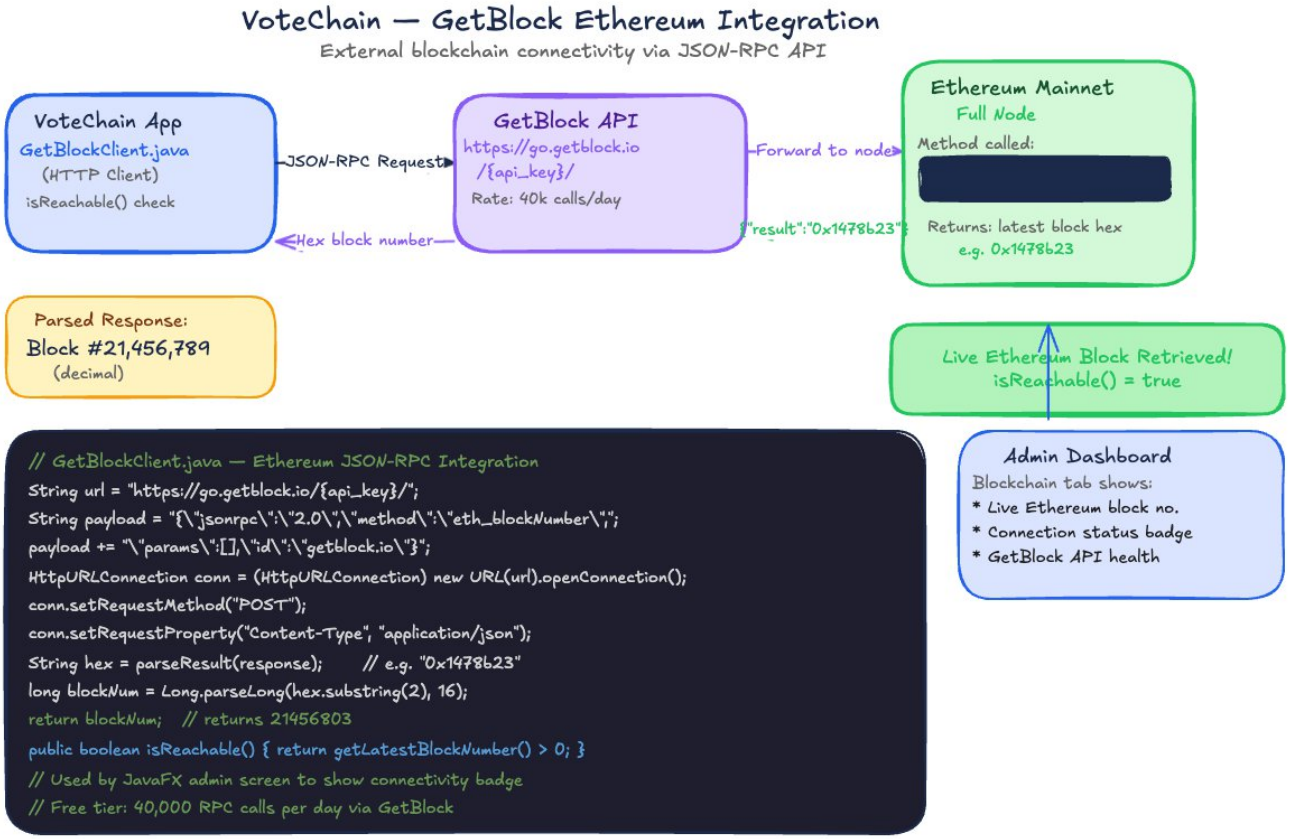

Figure 13: GetBlock — Ethereum Block Fetch Demo

8. BUSINESS USE CASES

Use Case 1: State Assembly Election (Vidhan Sabha)

Scenario: Maharashtra conducts its Vidhan Sabha election across 288 constituencies. The Election Commission deploys VoteChain to enable secure, blockchain-backed online voting.

How VoteChain Solves It: All 288 constituencies are pre-loaded with verified candidate lists. Voters authenticate using their Voter ID and are presented only their constituency's ballot. Each vote is mined into a new block with SHA-256 Proof-of-Work, creating an immutable record. Results are computed in real-time by traversing the blockchain — no manual counting required. The admin console provides constituency-level drilldowns and division-wise analytics.

Use Case 2: Tamper Detection & Audit

Scenario: After an election, allegations of result manipulation arise and an independent audit is demanded.

How VoteChain Solves It: Any authorized auditor accesses the admin console and clicks Validate Blockchain. The system recomputes every block's SHA-256 hash, verifies hash links, and checks Proof-of-Work compliance. If any block has been modified, validation immediately fails — identifying the exact tampered block with its index, expected hash, and actual hash — providing mathematical proof of tampering.

Use Case 3: Remote / Migrant Voter Participation

Scenario: Thousands of Maharashtra's migrant workers live in other states and cannot travel to their home constituency on election day.

How VoteChain Solves It: VoteChain's web interface allows authenticated voters to cast their vote from any device with internet access. Their vote is tied to their registered constituency regardless of physical location. Blockchain immutability ensures the vote cannot be altered in transit or after recording.

9. APPLICATION USE CASES

Use Case 1: Voter Casts a Vote

Field	Details
Actor	Registered Voter
Precondition	Voter is registered with a valid Voter ID, password, and constituency assignment.
Workflow	Step 1: Open the web interface and click Voter Login. Step 2: Enter Voter ID (e.g., MH001) and password. Step 3: System authenticates via BCrypt against MongoDB. Step 4: Voter Dashboard shows active elections. Step 5: Click Cast Vote for Vidhan Sabha. Step 6: System loads candidates for the voter's constituency. Step 7: Select a candidate and submit. Step 8: Backend mines a new block (PoW, difficulty = 3). Step 9: Block saved to MongoDB; voter marked as voted. Step 10: Confirmation screen shows block hash and mining time.
Expected Outcome	Vote is immutably recorded on the blockchain. Voter cannot vote again in the same election.

Use Case 2: Admin Validates Blockchain

Field	Details
Actor	Election Commission Administrator
Precondition	Admin is logged in with valid credentials. Votes have been cast.
Workflow	Step 1: Admin logs in (credentials: admin / admin123). Step 2: Navigate to the Blockchain tab. Step 3: View the complete chain timeline with block details. Step 4: System validates every block — hash integrity, chain linking, PoW compliance. Step 5: If valid, displays 'Blockchain Secure' status badge. Step 6: If tampered, displays TAMPERED alert with the corrupted block index, expected hash, and actual hash.
Expected Outcome	Admin has mathematical proof of chain integrity or can identify the exact corruption point.

Use Case 3: Admin Registers a New Voter

Field	Details
Actor	Election Commission Administrator
Precondition	Admin is logged in to the system.
Workflow	Step 1: Navigate to the Register tab in the Admin Dashboard. Step 2: Fill in Voter ID, Full Name, Password, Division, District, and VS Constituency using cascading dropdowns. Step 3: Click Register Voter. Step 4: System hashes the password with BCrypt (cost factor 12) and saves to MongoDB. Step 5: Success message confirms registration.
Expected Outcome	New voter can log in and cast their vote in any active election.

10. FUTURE SCOPE

#	Enhancement	Description
1	Distributed P2P Network	Deploy multiple blockchain nodes across districts enabling true decentralized consensus using protocols like PBFT or Raft.
2	Aadhaar-Based Biometric Auth	Integrate UIDAI's Aadhaar authentication API for fingerprint/iris-based voter verification, eliminating password-based login.
3	Smart Contracts (Solidity)	Deploy voting logic as Solidity smart contracts on a public or permissioned blockchain (e.g., Polygon, Hyperledger Fabric).
4	Mobile Application	Develop native Android/iOS apps with QR-code based voter authentication and offline vote queuing.
5	Zero-Knowledge Proofs	Implement ZK-SNARKs to verify vote validity without revealing individual voter choices, ensuring ballot secrecy at the protocol level.
6	Multi-State Expansion	Extend the data seeder and UI to support all 28 Indian states and 8 union territories with their full constituency structures.
7	AI Anomaly Detection	Use machine learning models to detect unusual voting patterns (e.g., sudden turnout spikes) in real-time and flag suspicious activity.
8	Homomorphic Encryption	Encrypt votes so that ballots can be tallied without decrypting individual votes, ensuring end-to-end privacy.
9	IPFS-Based Storage	Store candidate photos, party manifestos, and voter receipts on IPFS for decentralized, censorship-resistant access.
10	SMS/Email Notifications	Notify voters via SMS/Email when their vote is confirmed on the blockchain, providing an additional verification channel.

Table 10.1: Future Scope Enhancements

11. CHALLENGES FACED

11.1 Technical Challenges

Challenge	Resolution
Genesis Block Hash Inconsistency	Using LocalDateTime.now() in the genesis block constructor produced a different hash on every restart, breaking the chain link for Block #1 stored in MongoDB. Resolved by hardcoding the genesis timestamp to 2024-01-01 00:00:00.
BCrypt Integration with MongoDB	Stored password hashes are 60 characters with embedded salts. Verification required using BCrypt.checkpw() to extract the salt from the stored hash automatically.
CORS Issues	The web frontend was blocked from calling the Spark Java API. Resolved by adding a before() filter with Access-Control-Allow-Origin: * and handling OPTIONS preflight requests.
MongoDB Connection Pooling	Multiple service classes each created their own MongoClient, risking connection pool exhaustion. Mitigated by MongoDB driver's internal connection pooling mechanism.
Proof-of-Work Performance	Mining blocks with difficulty 4+ caused multi-second delays. Resolved by keeping difficulty at 3 for demo purposes with a configurable setDifficulty() method.
Fat JAR Packaging	JavaFX dependencies conflicted with the Maven Shade Plugin. Resolved by separating ApiOnlyMain (no JavaFX imports) as the JAR entry point for server-only deployment.

Table 11.1: Technical Challenges and Resolutions

11.2 Team / Management Challenges

- Balancing the complexity of real-world Maharashtra data (288 constituencies × 6+ candidates each) with available development time required careful sprint planning.
- Maintaining premium web frontend design quality while ensuring cross-browser compatibility across Chrome, Firefox, and Edge demanded extensive testing iterations.

11.3 Resource Limitations

- Free-tier MongoDB Atlas cluster limited to 512 MB storage and shared cluster performance, requiring efficient schema design.
- GetBlock free tier limited to 40,000 API calls per day, requiring careful usage throttling during testing.

12. OUTCOME

12.1 Results Achieved

Metric	Result
Total Lines of Code	6,723 LOC
Java Source Files	27 classes across 7 packages
API Endpoints	20+ REST endpoints
Constituencies Seeded	336 (288 VS + 48 LS)
Candidates Generated	~2,700 (8 per constituency)
Demo Voters	30 (spread across all 6 divisions)
Blockchain Validation Accuracy	100% — all valid chains pass, all tampered chains fail
Duplicate Vote Prevention	100% — no voter can cast more than one vote per election
Password Security	BCrypt cost factor 12 — no plaintext passwords stored
PoW Mining Time	10 – 50 ms per block (difficulty 3)
External Blockchain	Successfully retrieved live Ethereum block numbers via GetBlock

Table 12.1: Project Metrics and Results

12.2 Performance Improvements

- Real-time result computation: Results are computed by traversing the blockchain in O(n) time — no separate tallying step required.
- Cloud persistence: Votes are saved to MongoDB Atlas immediately after mining, eliminating local file dependency and single points of failure.
- Instant validation: Full chain validation completes in under 100 ms for chains with fewer than 1,000 blocks.

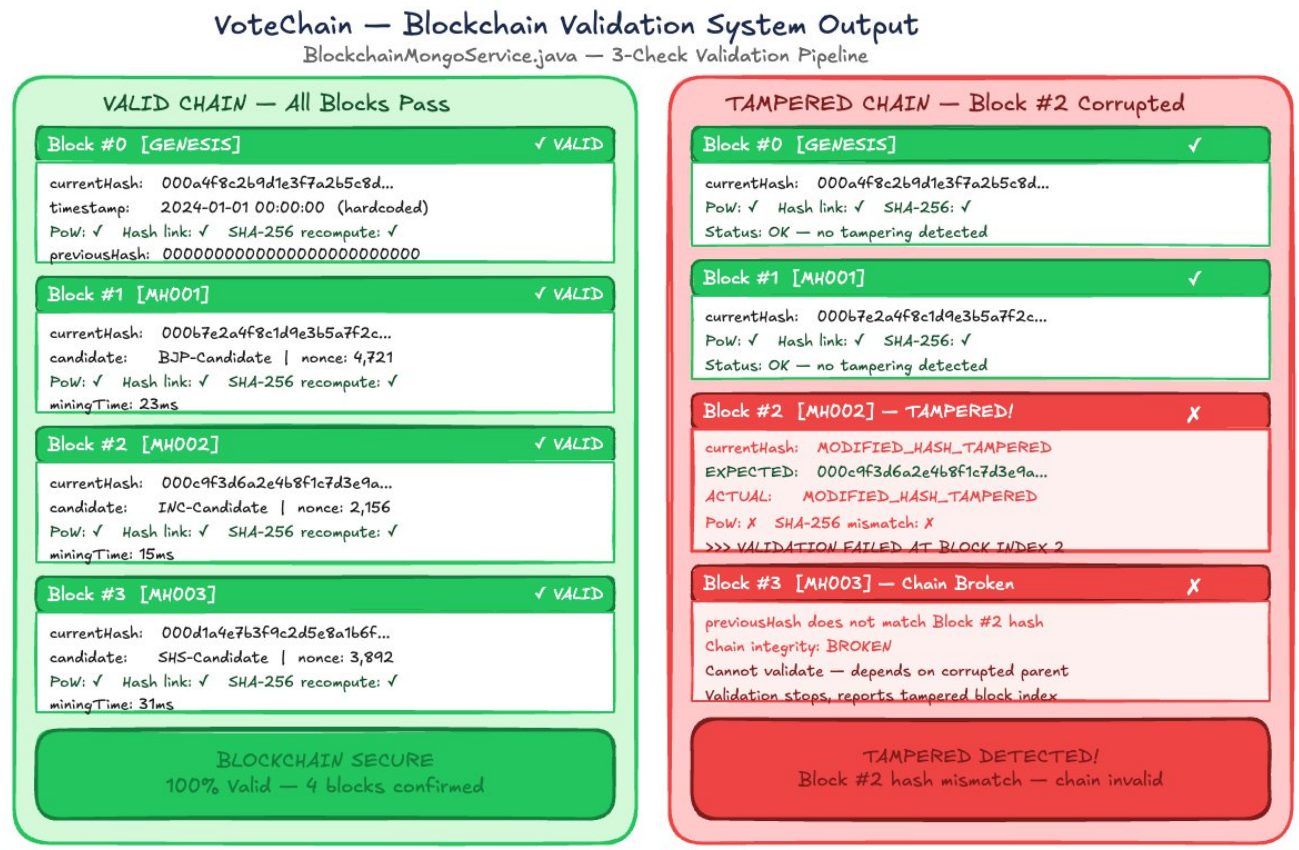

Figure 14: Blockchain Validation — System Output (Valid vs Tampered Chain)

Figure 14: Blockchain Validation — System Output

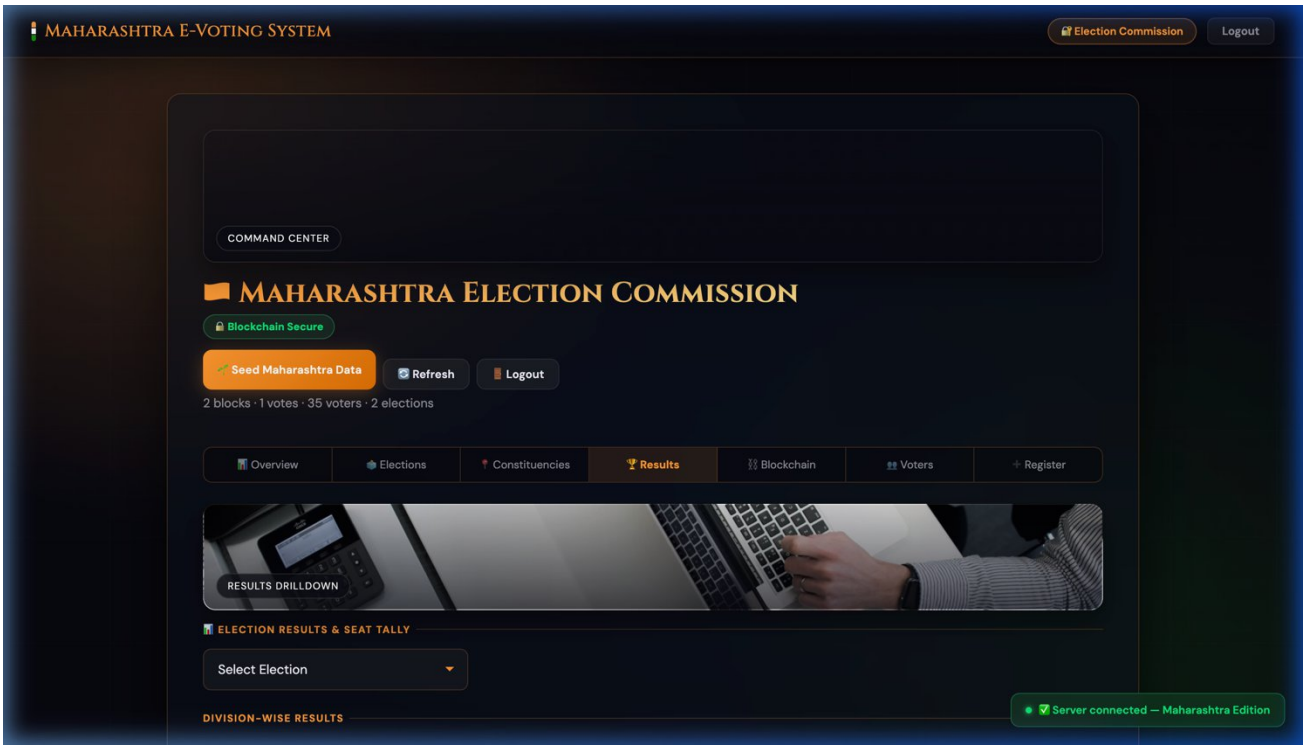

Figure 15: Election Results — Admin Dashboard View

13. CONCLUSION

13.1 Summary

VoteChain is a comprehensive, production-grade blockchain-based electronic voting system developed for Maharashtra state elections. The project successfully demonstrates how cryptographic techniques — SHA-256 hashing, Proof-of-Work mining, RSA-2048 digital signatures, and BCrypt password security — can be combined to create a transparent, tamper-evident, and publicly auditable voting system.

The system models real-world electoral data (288 Vidhan Sabha and 48 Lok Sabha constituencies across 6 divisions and 36 districts) with realistic candidate pools from 11 political parties. The dual-interface approach (web + desktop) ensures accessibility for all user types, while MongoDB Atlas provides reliable cloud-hosted persistence for votes and voter data.

13.2 Learning Outcomes

Through this project, the team developed the following skills and domain knowledge:

- Blockchain Architecture — Designing and implementing a hash-chained, Proof-of-Work blockchain from scratch in Java.
- Cryptographic Security — Applying SHA-256, RSA-2048, and BCrypt in a real-world security context.
- Full-Stack Development — Building a complete system spanning backend APIs, database integration, and frontend interfaces.
- REST API Design — Creating 20+ well-structured API endpoints with proper error handling, CORS, and JSON serialization.
- Cloud Database — Deploying and querying MongoDB Atlas in a production-like environment.
- Domain Modeling — Representing India's complex electoral structure in a scalable data model.
- External API Integration — Connecting to live Ethereum infrastructure via JSON-RPC.
- Software Engineering Practices — Applying layered architecture, separation of concerns, and Maven-based build management.

13.3 Final Impact

VoteChain proves that blockchain-based voting is not merely theoretical but can be built with existing, widely-available technologies. While the current implementation operates as a centralized chain on a single server, it establishes the architectural foundation upon which a fully distributed, peer-to-peer voting protocol can be built. The project serves as both an educational demonstration and a practical prototype for modernizing India's electoral infrastructure.

14. REFERENCES

1. Satoshi Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. https://bitcoin.org/bitcoin.pdf
2. Election Commission of India, Delimitation of Constituencies — Maharashtra. https://eci.gov.in/
3. Oracle, Java SE 17 Documentation — java.security.MessageDigest (SHA-256). https://docs.oracle.com/
4. MongoDB, Inc., MongoDB Atlas — Cloud-Hosted NoSQL Database. https://www.mongodb.com/atlas
5. Niels Provos and David Mazieres, A Future-Adaptable Password Scheme (BCrypt), USENIX, 1999.
6. Spark Java Framework — Simple and lightweight Java web framework. https://sparkjava.com/
7. GetBlock — Blockchain RPC Node Provider. https://getblock.io/
8. Chart.js — Simple yet flexible JavaScript charting library. https://www.chartjs.org/
9. OpenJFX Documentation, JavaFX 17 API Documentation. https://openjfx.io/
10. Vitalik Buterin, Ethereum White Paper, 2014. https://ethereum.org/en/whitepaper/
11. NIST, Secure Hash Standard (SHA-256), FIPS 180-4. https://csrc.nist.gov/
12. Anton Badev and Matthew Chen, Bitcoin: Technical Background and Data Analysis, Federal Reserve Board, 2014.

15. APPENDIX

A. Block Data Structure (Java)

```
public class Block implements Serializable {     private int index;     private String voterId;     private String candidateName;     private String timestamp;     private String previousHash;     private String currentHash;     private int nonce;     private long miningTime;     private String digitalSignature;     private String publicKey;     private static int difficulty = 3;     public String calculateHash() {         String data = index + voterId + candidateName                    + timestamp + previousHash + nonce;         return HashUtil.calculateSHA256(data);     } }
```

B. MongoDB Collections Schema

Collection	Key Fields
votingdb.blocks	index, voterId, candidateName, timestamp, previousHash, currentHash (starts with '000'), nonce, miningTime, electionId, constituencyId, party
votingdb.voters	voterId, name, password (BCrypt 60-char hash), district, division, constituencyVS, constituencyLS, hasVoted, votedElections
votingdb.elections	electionId, type (VIDHAN_SABHA / LOK_SABHA), title, state, totalSeats, status (UPCOMING / ACTIVE / COMPLETED), startDate, endDate
votingdb.constituencies	constituencyId, electionId, name, number, type, district, division, reservationCategory (GENERAL / SC / ST), candidates[]

Table 7.2: MongoDB Collections Schema

C. Default Demo Credentials

Role	ID	Password
Administrator	admin	admin123
Demo Voter 1	MH001	voter1
Demo Voter 2	MH002	voter2
Demo Voter 30	MH030	voter30
Legacy Voter	V001	voter1

D. Project File Structure (Abbreviated)

```
MPJ/ ├── src/main/java/com/votingsystem/ │   ├── api/         # Spark Java REST API server & routing mappings │   ├── integration/ # GetBlock / Ethereum RPC integration │   ├── model/       # Block, Voter, Election, Constituency entities │   ├── security/    # BCrypt and SHA-256 hashing utilities │   ├── service/     # Core business logic & blockchain rules │   ├── storage/     # MongoDB Atlas DAO connections │   └── ui/          # JavaFX desktop application screens ├── voting_index.html   # Main Web Application Frontend ├── app.js              # Web Application Logic & API Fetch Calls ├── styles.css          # Web Application Styles ├── pom.xml             # Maven Dependencies & Build Configuration ├── Dockerfile          # Container deployment configuration └── README.md           # Project Documentation
```

— End of Report —

Department of Computer Science Engineering and Technology
Dr. Vishwanath Karad MIT World Peace University, Pune | Academic Year: 2025–2026